



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

UNIDAD 2

Objetos, clases, herencia y polimorfismo.

Tema:

FUNDAMENTOS DE CONSTRUCTORES Y DESTRUCTORES





RESULTADO DE APRENDIZAJE

Implementa programas que hacen uso de relación de herencia, interfaces y clases abstractas con métodos y variables polimórficas.

Al finalizar la sesión, el estudiante será capaz de:

1

Comprender el funcionamiento del constructor (`__init__`) y del destructor (`__del__`) dentro del ciclo de vida de un objeto.

2

Aplicar constructores para inicializar correctamente los modelos aplicado en proyectos modernos con Python.

3

Reconocer el uso del decorador `@dataclass` como una alternativa moderna para generar automáticamente constructores en Python.

INTRODUCCIÓN

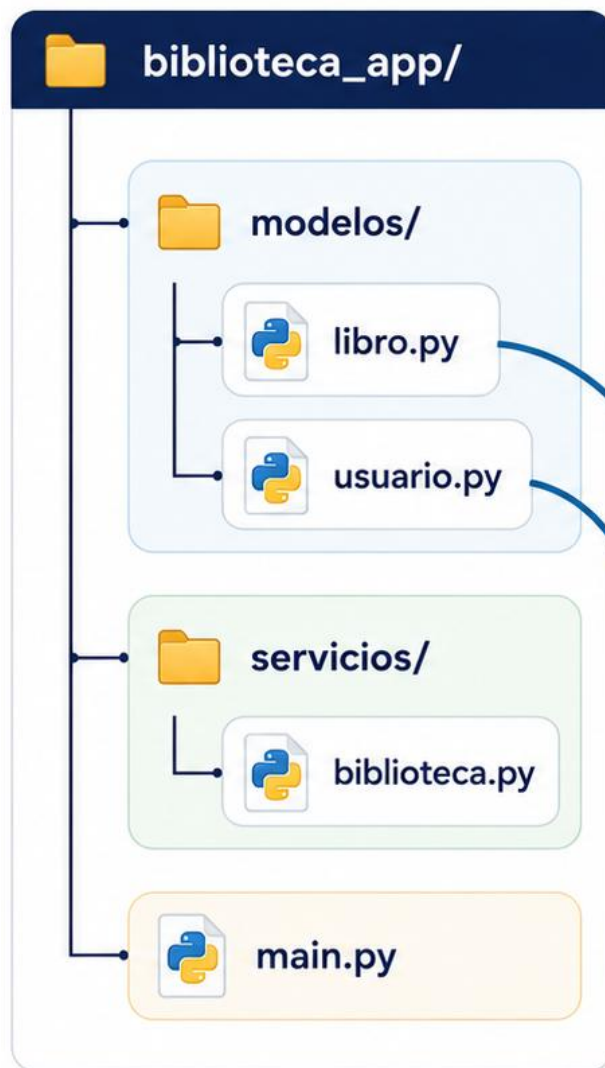
Hasta este momento hemos aprendido a definir clases, crear objetos, organizar proyectos mediante una arquitectura modular y aplicar los principales fundamentos de la Programación Orientada a Objetos.

➡ Sin embargo, todavía existe una pregunta importante:
¿Cómo garantizamos que cada objeto nazca correctamente inicializado antes de comenzar a trabajar con él?

Durante esta sesión conoceremos el papel que desempeñan los constructores dentro del ciclo de vida de un objeto, comprenderemos cuándo intervienen los destructores y descubriremos cómo Python moderno simplifica este proceso mediante el uso del decorador @dataclass.



RECORDANDO NUESTRA ARQUITECTURA



Nuestro proyecto se ha organizado de forma **modular** para separar responsabilidades y facilitar el mantenimiento del código.

MODELOS



Representan las entidades del sistema, como:

- Libro
- Usuario

SERVICIOS



Administran las operaciones del sistema, como:

- Registrar
- Listar
- Buscar información

ARCHIVO PRINCIPAL



Permite ejecutar el programa y coordinar la interacción entre los componentes.



Constructor:

inicializa los atributos del objeto al momento de crear una instancia.



IDEA CLAVE

El constructor se ubica principalmente **dentro de cada clase de modelo**, porque allí **se inicializan los atributos que describen a cada objeto**.

¿QUÉ ES UN CONSTRUCTOR?



Un **constructor** es un método especial que se ejecuta automáticamente cuando se crea una nueva instancia de una clase.



En Python, el constructor se define mediante el método `__init__`. Su función principal es **inicializar los atributos** del objeto, asignando los valores iniciales que necesita para comenzar a trabajar correctamente.



Gracias al constructor, cada objeto puede nacer con un **estado definido**. Esto evita que el programa utilice objetos incompletos o con información no inicializada.



IDEA CLAVE



El constructor responde a la pregunta:

¿con qué información debe iniciar este objeto?

1

Crear objeto



El programador solicita crear una nueva instancia de la clase.

2

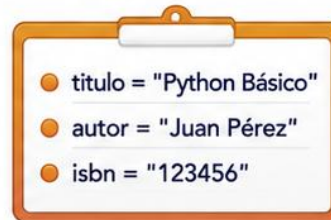
Se ejecuta `__init__`



Python ejecuta automáticamente el método especial `__init__`, el constructor de la clase.

3

Se asignan atributos



El constructor asigna los valores iniciales a cada atributo del objeto.

4

Objeto listo para utilizarse



El objeto queda completamente inicializado y listo para ser utilizado en el programa.

CONSTRUCTOR APLICADO A LA CLASE LIBRO



En el proyecto **Biblioteca**, un libro no debería crearse vacío.



Cada objeto de tipo **Libro** debe iniciar con información básica: **título**, **autor** e **ISBN**.



El constructor permite recibir estos datos al momento de crear el objeto y almacenarlos como atributos propios de la instancia.

Clase Libro



Libro



título

Atributo que almacena el título del libro.



autor

Atributo que almacena el autor del libro.



isbn

Atributo que almacena el ISBN del libro.



Constructor de la clase Libro

```
class Libro:  
  
    def __init__(self, titulo, autor, isbn):  
        self.titulo = titulo  
        self.autor = autor  
        self.isbn = isbn
```



¿Qué hace?

El método `__init__` recibe los datos necesarios al crear un objeto **Libro** y asigna esos valores a los atributos de la instancia.

libro1



título: "Python Básico"
autor: "Juan Pérez"
isbn: "978-1-234567-00-1"



Este objeto **Libro** tiene sus propios valores.

≠

Información diferente

libro2



título: "POO con Python"
autor: "Ana López"
isbn: "978-1-234567-00-2"



Este objeto **Libro** tiene sus propios valores.



IDEA CLAVE



Cada objeto **Libro** tendrá sus **propios valores**, aunque todos compartan la misma estructura de clase.

DEL MAIN.PY AL CONSTRUCTOR



El constructor no se ejecuta de forma aislada.

Se activa desde `main.py` cuando el programa crea un objeto de una clase.



En nuestro proyecto, `main.py` crea un objeto `Libro`, Python ejecuta automáticamente `__init__`, y luego ese objeto puede ser enviado al servicio `Biblioteca` para ser gestionado.



`main.py`

```
from modelos.libro import Libro
from servicios.biblioteca import Biblioteca

biblioteca = Biblioteca()

libro1 = Libro("Python Básico", "Ana Torres", "ISBN-001")
biblioteca.agregar_libro(libro1)
```

1

`main.py`



El programa inicia en `main.py`



2

Se crea el objeto `Libro` con los datos necesarios.

```
Libro(
    "Python Básico",
    "Ana Torres",
    "ISBN-001"
)
```

3

`modelos/libro.py`

Python ejecuta automáticamente el constructor `__init__`.



`__init__`

Se inicializan los atributos del objeto `Libro`.



Objeto `Libro` completamente inicializado

4

El objeto `Libro` ya tiene sus atributos listos para usarse.



`libro1`

5

`servicios/biblioteca.py`

El objeto se envía al servicio `Biblioteca` para ser gestionado.



`Biblioteca`

`agregar_libro(libro1)`



El libro se registra en la colección interna del servicio.



IDEA CLAVE

El **constructor** permite que el **objeto** llegue **completo** (con sus datos iniciales) al **servicio** que lo va a administrar.



CONSTRUCTORES CON PARÁMETROS Y VALORES PREDETERMINADOS



Los constructores pueden recibir **parámetros obligatorios** y **valores predeterminados**.



Esto permite crear objetos con la información **mínima necesaria** y, al mismo tiempo, definir **valores iniciales** cuando un dato no se proporciona directamente.



En una biblioteca, un libro nuevo puede iniciar como **disponible** de forma **predeterminada**.



Constructor con parámetros y valor predeterminado

```
class Libro:
```

```
    def __init__(self, titulo, autor, isbn, disponible=True):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn
        self.disponible = disponible
```



Si no se envía **disponible**, Python usa **True**.



IDEA CLAVE



Los valores predeterminados ayudan a crear objetos **más flexibles** sin perder claridad.



PARÁMETROS OBLIGATORIOS

Debe proporcionarlos **siempre** al crear el objeto.



titulo

Título del libro.

Ejemplo:
"Python Básico"



autor

Autor del libro.

Ejemplo:
"Ana Torres"



isbn

Identificador único del libro.

Ejemplo:
"ISBN-001"



PARÁMETRO CON VALOR PREDETERMINADO

Tiene un **valor por defecto** si no se envía.



disponible = True

Indica si el libro puede ser prestado.

Si no se envía:



Python usa
True

Si se envía **False**:



El libro inicia como
no disponible.

EJEMPLOS

Libro (sin enviar disponible)



Libro("Python Básico",
"Ana Torres", "ISBN-001")

→ disponible = True



Libro (enviando disponible=False)



Libro("Python Básico",
"Ana Torres", "ISBN-001", False)

→ disponible = False



BUENAS PRÁCTICAS EN EL CONSTRUCTOR



Un constructor debe ser **claro, directo y fácil de mantener**.

Su responsabilidad principal es **preparar el estado inicial** del objeto, no resolver toda la lógica del sistema.



Cuando el constructor está bien diseñado, el resto del programa puede trabajar con objetos más **confiables y predecibles**.



Estas prácticas permiten crear **código más limpio, organizado** y preparado para **crecer**.



BUENAS PRÁCTICAS EN EL CONSTRUCTOR



Sigue estas recomendaciones para escribir constructores **claros, flexibles y fáciles de mantener**.

1



Inicializar todos los atributos importantes del objeto.
Asegura que el objeto nazca en un estado completo y consistente.

2



Utilizar nombres descriptivos para los parámetros.
Mejora la comprensión del código y facilita su uso por otros desarrolladores.

3



Usar valores predeterminados solo cuando tengan sentido dentro del problema.
No todos los atributos necesitan un valor por defecto.

4



Evitar lógica extensa o procesos complejos dentro del constructor.
La lógica compleja debe manejarse en métodos de la clase o en los servicios.

5



Mantener una sola responsabilidad: dejar el objeto listo para trabajar.
El constructor prepara, no ejecuta funciones del sistema.



BUEN EJEMPLO

```
class Libro:
    def __init__(self, titulo, autor, isbn, disponible=True):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn
        self.disponible = disponible
```

Claridad, inicialización y valor predeterminado correcto.



EJEMPLO A EVITAR

```
class Libro:
    def __init__(self, t, a, i):
        self.t = t
        self.a = a
        self.i = i
        self.guardar_en_base_de_datos() # X Lógica extensa
        self.validar_datos()           # X No corresponde aquí
```

Poco claro, sin valores útiles y con lógica compleja.



IDEA CLAVE



Un buen constructor **evita errores futuros** y **mejora la calidad** del diseño orientado a objetos.

CONTROL DE ATRIBUTOS CON @PROPERTY



Después de inicializar un objeto mediante el constructor, también es importante **controlar cómo se accede** a ciertos atributos.



En otros lenguajes se suelen crear métodos llamados **getter y setter**.



En Python, una forma más moderna y legible de hacerlo es mediante propiedades, utilizando el decorador **@property**.



Una propiedad permite acceder a un método como si fuera un atributo, manteniendo una **sintaxis sencilla** para quien utiliza el objeto.

CONTROL DE ATRIBUTOS CON @PROPERTY



1. CONSTRUCTOR

El constructor inicializa los atributos del objeto.



2. ATRIBUTO INTERNO

El valor se almacena en un atributo interno (convención con guion bajo).
_titulo



3. @PROPERTY

El método decorado con @property controla el acceso y retorna el valor deseado.



4. ACCESO CONTROLADO

El atributo se consulta como si fuera un atributo normal: **libro.titulo**



EJEMPLO EN CÓDIGO

```
class Libro:
    def __init__(self, titulo, autor, isbn):
        self._titulo = titulo
        self.autor = autor
        self.isbn = isbn

    @property
    def titulo(self):
        return self._titulo
```



¿QUÉ PERMITE?

Acceder al título del libro como si fuera un atributo público, pero con la ventaja de poder **controlar** qué valor se devuelve.

¿CÓMO SE USA?



IDEA CLAVE

@property permite consultar un valor de forma **controlada** sin cambiar la forma natural de usar el objeto.



ACTUALIZAR DATOS CON @SETTER



ACTUALIZAR DATOS CON @SETTER



El decorador **@setter** permite **controlar cómo se modifica** un atributo después de que el objeto ya fue creado.



Esto es útil para **validar los datos antes de actualizar** la información interna del objeto.



Por ejemplo, evitamos que un libro tenga un **título vacío**.



IDEA CLAVE

El setter permite **proteger el estado del objeto** cuando sus datos cambian.

¿CÓMO FUNCIONA EL @SETTER?



Nuevo título
asignado al objeto
`libro.titulo = "Nuevo título"`



@titulo.setter
se ejecuta automáticamente



¿El nuevo título está vacío?
`nuevo_titulo.strip() == ""`



Sí, está vacío



ERROR

`raise ValueError("El título no puede estar vacío")`



No se actualiza
el atributo interno
`_titulo` mantiene
su valor anterior



NO, tiene contenido



VÁLIDO

Se actualiza el atributo
interno del objeto

`_titulo = nuevo_titulo`
El objeto se mantiene
consistente y protegido



EJEMPLO EN CÓDIGO

```

class Libro:
    def __init__(self, titulo, autor, isbn):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn

    @property
    def titulo(self):
        return self._titulo

    @titulo.setter
    def titulo(self, nuevo_titulo):
        if nuevo_titulo.strip() == "":
            raise ValueError("El título no puede estar vacío")
        self._titulo = nuevo_titulo
  
```

EJEMPLOS DE USO

Intento con título vacío

`libro.titulo = ""`
Nuevo título vacío

@titulo.setter
valida

Error lanzado
`ValueError`
("El título no puede estar vacío")

Intento con título válido

`libro.titulo = "Python Avanzado"`
Nuevo título válido

@titulo.setter
valida

Actualiza _titulo
`_titulo = "Python Avanzado"`



El **setter** es el complemento de **@property**: juntos permiten un **acceso controlado y seguro**.



PYTHON MODERNO: @DATACLASS

PYTHON MODERNO: @DATACLASS



Python ofrece herramientas modernas que permiten **reducir código repetitivo** cuando una clase se utiliza principalmente para almacenar datos.



El decorador **@dataclass** permite declarar una clase de forma más compacta.

A partir de los atributos indicados, Python genera automáticamente un constructor y otros métodos útiles.



Esto no elimina la importancia de aprender **__init__**.

Primero debemos comprender cómo funciona el **constructor tradicional**; luego podemos reconocer cuándo conviene usar una alternativa más moderna.



@dataclass es ideal para clases que representan datos.

Si la clase tiene lógica compleja, validaciones o comportamientos especiales, el **constructor tradicional** sigue siendo la mejor opción.

CONSTRUCTOR TRADICIONAL



Definimos el constructor manualmente y escribimos más código.

```
class Libro:
    def __init__(self, titulo, autor, isbn, disponible=True):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn
        self.disponible = disponible

    # Métodos adicionales (si se necesitan)
    def __str__(self):
        return f"{self.titulo} - {self.autor}"
```



Python genera el constructor automáticamente

CON @DATACLASS



Declaramos solo los atributos y Python genera el resto por nosotros.

```
from dataclasses import dataclass

@dataclass
class Libro:
    titulo: str
    autor: str
    isbn: str
    disponible: bool = True
```

¿QUÉ OBTENEMOS?



Escribimos el constructor



Mayor control para validaciones



Ideal para lógica compleja



Más flexible en comportamiento

¿QUÉ OBTENEMOS AUTOMÁTICAMENTE?



Constructor **__init__()**



__repr__() legible



__eq__() comparación



Y otros métodos útiles

COMPARACIÓN RÁPIDA



- Más código escrito a mano.
- Mayor control y flexibilidad.
- Ideal para lógica compleja y validaciones.



- Menos código repetitivo.
- Constructor y métodos generados automáticamente.
- Ideal para clases que solo representan datos.



IDEA CLAVE

@dataclass ayuda a escribir **menos código** cuando la clase se orienta principalmente a representar datos, pero el **constructor tradicional** sigue siendo esencial para comprender la POO a profundidad.



¿CUÁNDO USAR `__init__` Y CUÁNDO `@dataclass`?



Tanto `__init__` como `@dataclass` permiten trabajar con constructores, pero no se utilizan exactamente con el mismo propósito.



El constructor tradicional ofrece **mayor control** y es recomendable cuando necesitamos validaciones, procesos específicos o lógica personalizada al crear el objeto.



`@dataclass` es útil cuando la clase representa **principalmente datos** y no requiere una lógica compleja de inicialización.

CONSTRUCTOR TRADICIONAL `__init__`



Mayor control sobre la inicialización. Puedes definir exactamente cómo se crea y configura el objeto.



Permite escribir validaciones desde el inicio. Puedes asegurarte de que los datos que recibe el objeto sean correctos.



Adecuado para clases con lógica personalizada. Ideal cuando se requieren procesos especiales al momento de crear el objeto.



Es la base conceptual que todo estudiante debe dominar. Comprender `__init__` es comprender la POO desde su fundamento.



Ambas opciones son válidas.

La elección depende del **objetivo de tu clase.**

DECORADOR `@dataclass`



Reduce código repetitivo. Solo declaras los atributos y Python se encarga del resto.



Genera automáticamente el constructor. Python crea `__init__` y otros métodos útiles por ti.



Adecuado para clases orientadas principalmente a datos. Perfecto para estructuras simples sin lógica de inicialización compleja.



Es una herramienta moderna que se comprende mejor después de aprender `__init__`. Permite escribir menos código sin perder claridad.



IDEA CLAVE

En esta clase usaremos `__init__` como **fundamento principal** y `@dataclass` como una **alternativa moderna** para reconocer y aplicar progresivamente.



¿QUÉ ES UN DESTRUCTOR?



¿QUÉ ES UN DESTRUCTOR?



Un destructor es un **método especial** que puede ejecutarse cuando un objeto está por ser eliminado.



En Python, el destructor se define mediante el método `__del__`.



Su finalidad **no es liberar memoria** manualmente, porque Python ya cuenta con un **recolector de basura** que administra la memoria automáticamente.



El destructor puede ser útil en casos específicos donde el objeto trabaja con **recursos externos**, como archivos abiertos o conexiones que deben cerrarse correctamente.

¿CÓMO FUNCIONA?

1



Objeto en uso
El objeto se crea y se utiliza normalmente en el programa.



2



Deja de utilizarse
El programa ya no tiene referencias activas a ese objeto.



3



Recolector de basura
Python detecta que el objeto puede ser eliminado y lo marca para limpieza.



4



`__del__`
Si existe, Python ejecuta el método destructor del objeto.



5



Limpieza de recursos
Se liberan o cierran los recursos externos asociados al objeto.



EJEMPLO EN CÓDIGO

```
class RegistroArchivo:
    def __init__(self, nombre_archivo):
        self.archivo = open(nombre_archivo, "w")

    def __del__(self):
        self.archivo.close()

# Cuando el objeto sea eliminado por el recolector
# de basura, se ejecutará __del__ y se cerrará
# el archivo.
```



ADVERTENCIA

No depender siempre del destructor para cerrar recursos críticos.

- El momento exacto en que se ejecuta `__del__` no está garantizado.
- En algunos casos, puede ejecutarse tarde o incluso no ejecutarse.
- Es mejor cerrar los recursos manualmente (por ejemplo, con `close()`, `with`, etc.).



IDEA CLAVE

En Python, `__del__` existe, pero debe **usarse con cuidado** y **solo cuando tenga sentido** liberar recursos externos.



CONSTRUCTOR Y DESTRUCTOR EN EL CICLO DE VIDA DEL OBJETO





En esta semana, el proyecto **Biblioteca** mejora porque las clases ya no solo representan entidades del sistema, sino que también definen cómo deben **inicializarse correctamente sus objetos**.



La clase **Libro** puede iniciar con título, autor, ISBN y disponibilidad.



La clase **Usuario** puede iniciar con nombre, correo e identificador.



El servicio **Biblioteca** puede recibir esos objetos ya preparados para registrarlos, listarlos o gestionarlos.



Clase Libro

```
class Libro:
    def __init__(self, titulo, autor, isbn, disponible=True):
        self.titulo = titulo
        self.autor = autor
        self.isbn = isbn
        self.disponible = disponible
```

Ejemplo de creación

```
libro = Libro("Python Básico", "Ana Torres", "ISBN-001")
```



Servicio Biblioteca

```
class Biblioteca:
    def __init__(self):
        self._libros = []
        self._usuarios = []

    def agregar_libro(self, libro):
        self._libros.append(libro)

    def agregar_usuario(self, usuario):
        self._usuarios.append(usuario)

    def listar_libros(self):
        ...

    def listar_usuarios(self):
        ...
```



Libro
(objeto listo)



Usuario
(objeto listo)



Biblioteca
(registra y gestiona)



Listar, consultar,
actualizar, eliminar



Clase Usuario

```
class Usuario:
    def __init__(self, nombre, correo, id_usuario):
        self.nombre = nombre
        self.correo = correo
        self.id_usuario = id_usuario
```

Ejemplo de creación

```
usuario = Usuario("Juan Pérez", "juan@gmail.com", 1)
```



CONTROL CON PROPIEDADES

Permite controlar el acceso y la modificación de ciertos atributos para mantener la integridad de los datos.

```
libro.titulo # consulta controlada
libro.titulo = "Nuevo título" # validación antes de actualizar
```



PYTHON MODERNO: @DATACLASS

Ofrece una forma moderna de crear clases orientadas a datos con menos código repetitivo.

```
@dataclass
class Libro:
    titulo: str
    autor: str
    isbn: str
    disponible: bool = True
```



PROYECTO EN EVOLUCIÓN

Con estos fundamentos, podrás seguir mejorando el sistema en las próximas semanas, agregando nuevas estructuras y funcionalidades sin perder organización.



IDEA CLAVE

Aplicar constructores, propiedades, @setter, @dataclass y comprender el ciclo de vida del objeto hace que las clases sean más **claras, seguras y fáciles de mantener** dentro del sistema **Biblioteca**.



Listo para la práctica:
en la siguiente actividad usaremos estos conceptos en tu proyecto **Biblioteca**.

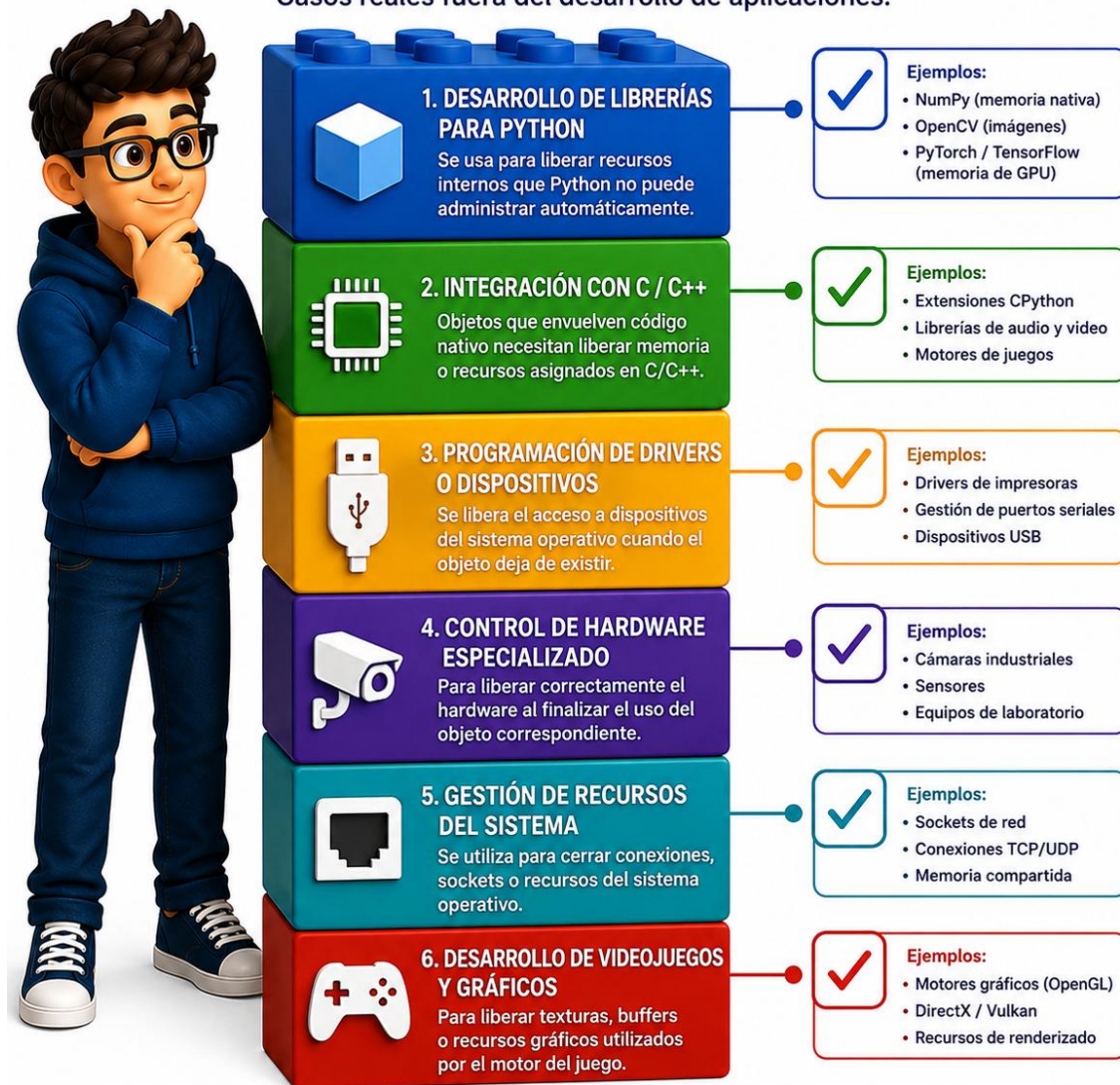
REFLEXIÓN

El constructor (`__init__`) es una herramienta fundamental para crear objetos correctamente inicializados y forma parte del desarrollo diario de aplicaciones en Python. En cambio, el destructor (`__del__`) continúa existiendo como parte del lenguaje, pero actualmente su uso es poco frecuente, ya que Python y los frameworks modernos ofrecen mecanismos más seguros para administrar los recursos.

Como desarrollador, no solo es importante conocer las herramientas que ofrece un lenguaje, sino también comprender cuándo y por qué utilizarlas.

¿DÓNDE SE UTILIZA ACTUALMENTE EL DESTRUCTOR (`__del__`)?

Casos reales fuera del desarrollo de aplicaciones.





Conclusiones

- El constructor `__init__` inicializa los atributos necesarios para que un objeto comience a trabajar correctamente.
- `@property` y `@setter` permiten controlar el acceso y modificación de ciertos atributos.
- `@dataclass` representa una alternativa moderna para crear clases orientadas a datos de forma más compacta.
- El destructor `__del__` debe comprenderse como un mecanismo de limpieza para casos específicos, no como una herramienta de uso obligatorio en todas las clases.

Hernández Bejarano, M., & Baquero Rey, L. E. (2023). Programación orientada a objetos en Java: Buenas prácticas. Ediciones de la U.

Hernández Bejarano, M., & Baquero Rey, L. E. (2025). Python con orientación a objetos y al análisis de datos. Ediciones de la U.

Bibliografía

Boucheny, V. (2025). Aprender programación orientada a objetos con Python (3.^a ed.).

Ediciones ENI. Piñeiro Gómez, J. M. (2022). Principios de la programación orientada a objetos. Paraninfo.



UEA
UNIVERSIDAD
ESTATAL AMAZÓNICA

JUNTOS TRANSFORMAMOS EL MUNDO DESDE LA AMAZONÍA



**CARRERAS
EN LÍNEA**

***Dr. David Sancho, PhD.
RECTOR UEA***